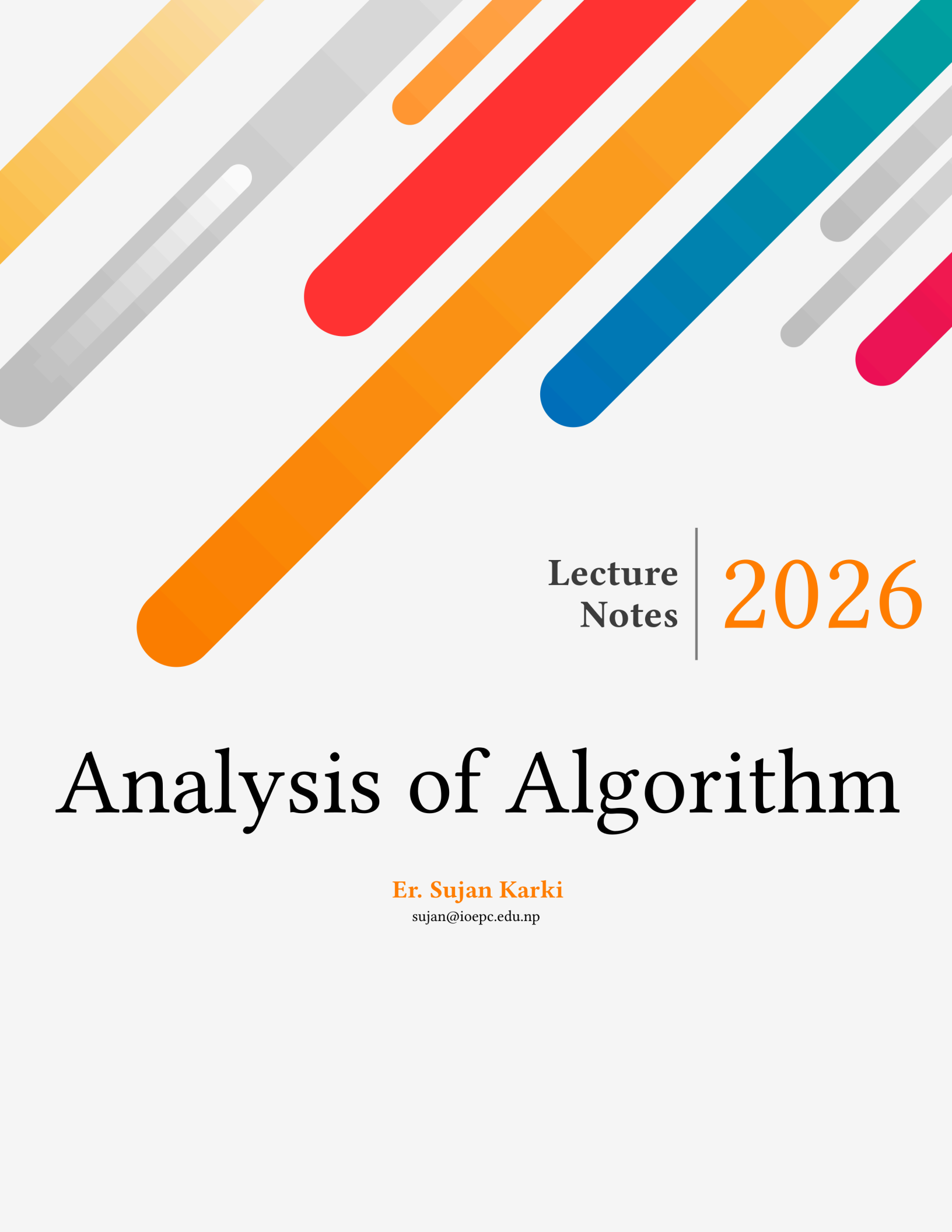




Lecture
Notes

2026

Analysis of Algorithm

Er. Sujan Karki

sujan@ioepc.edu.np

Chapter 3

Divide and Conquer Algorithms

Definition 3.0.1

A divide and conquer algorithm is a method used in computer science to solve big problems by breaking them down into smaller, more manageable parts. The idea is to “divide” the problem into smaller pieces, “conquer” each piece by solving it, and then “combine” the solutions to get the final answer.

Example 3.0.1

Imagine you have a big jigsaw puzzle with 1,000 pieces. Instead of trying to solve the whole puzzle at once, you could divide it into smaller sections, like the corners, edges, and middle. You solve each section separately, and then you combine them to complete the entire puzzle. This approach is similar to how a divide and conquer algorithm works.

3.0.1 Three Parts of Divide and Conquer

Divide-and-Conquer algorithms have three parts as follows:

1. **Divide** the problem into a number of sub-problems that are small instances of the main problem.
2. **Conquer** the sub-problems by solving them recursively. If they are small enough, solve the sub-problems as base cases.
3. **Combine** the solutions of the sub-problems as one complete solution for the main problem.

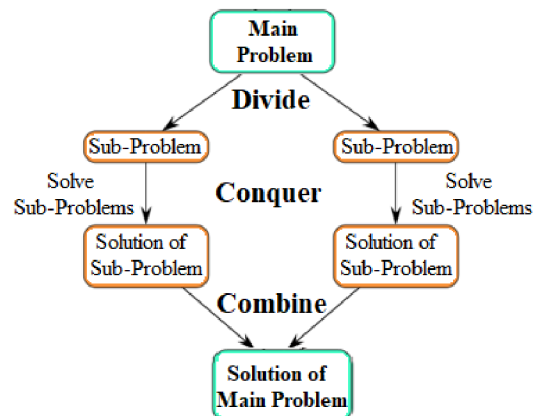


Figure 3.1: Diagrammatic Representation of General Procedure

3.1 Binary Search

Definition 3.1.1

Recursive Binary Search is a divide-and-conquer algorithm that finds the position of a target value within a **sorted array**. It works by:

- Comparing the target with the **middle element** of the array.
- If they are equal, the search is successful.
- If the target is smaller, the search continues recursively on the **left half** (elements before the middle).
- If the target is larger, the search continues recursively on the **right half** (elements after the middle).
- The recursion stops when the subarray has no elements (target not found).

Because the search space is halved at each step, the algorithm runs in $O(\log n)$ time.

Algorithm 3.1.1

1. Start
2. If $low > high$, return -1 (base case: target not found)
3. Compute $mid = low + \left\lfloor \frac{high - low}{2} \right\rfloor$ (integer division)
4. If $array[mid] = target$, return mid (target found)
5. If $target < array[mid]$, return $binarySearchRecursive(array, low, mid - 1, target)$ (search left half)
6. Else (i.e., $target > array[mid]$), return $binarySearchRecursive(array, mid + 1, high, target)$ (search right half)
7. Stop

Code

```

1 def binary_search_recursive(arr, target, low, high):
2     """
3     Recursive binary search.
4
5     Parameters:
6         arr (list): Sorted list of elements.
7         target: Element to search for.
8         low (int): Left boundary index.
9         high (int): Right boundary index.
10
11     Returns:
12         int: Index of target if found, otherwise -1.
13     """
14     # Base case: search space is empty
15     if low > high:
16         return -1
17
18     # Calculate middle index (avoid overflow)
19     mid = low + (high - low) // 2
20
21     # Target found

```

```

22     if arr[mid] == target:
23         return mid
24
25     # Search left half
26     elif target < arr[mid]:
27         return binary_search_recursive(arr, target, low, mid - 1)
28
29     # Search right half
30     else:
31         return binary_search_recursive(arr, target, mid + 1, high)

```

3.1.1 Time Complexity of Binary Search

Best Time Complexity

The best case scenario of Binary Search occurs when the target element is at the **central index**. In this situation, only one comparison is needed. Therefore, the best case time complexity of Binary Search is

$$O(1)$$

Worst Time Complexity

The worst case scenario of Binary Search occurs when the target element is not present in the array or is located at one of the ends. In this case, the search space is halved repeatedly until it becomes empty. The number of comparisons made in the worst case is proportional to the number of times the array can be halved, which is $\log_2 n$. Therefore, the worst case time complexity of Binary Search is

$$O(\log n)$$

1. Let the initial array have n elements.
2. After the first comparison, the search space reduces to at most $\frac{n}{2}$ elements.
3. After the second comparison, the search space reduces to at most $\frac{n}{4}$ elements.
4.
5. After k comparisons, the search space reduces to at most $\frac{n}{2^k}$ elements.
6. The search continues until the search space becomes empty (i.e., no elements left to examine). This happens when:

$$\frac{n}{2^k} < 1 \implies n < 2^k \quad (\text{taking logarithm on both sides}) \implies k > \log_2 n.$$

7. The smallest integer k satisfying this is:

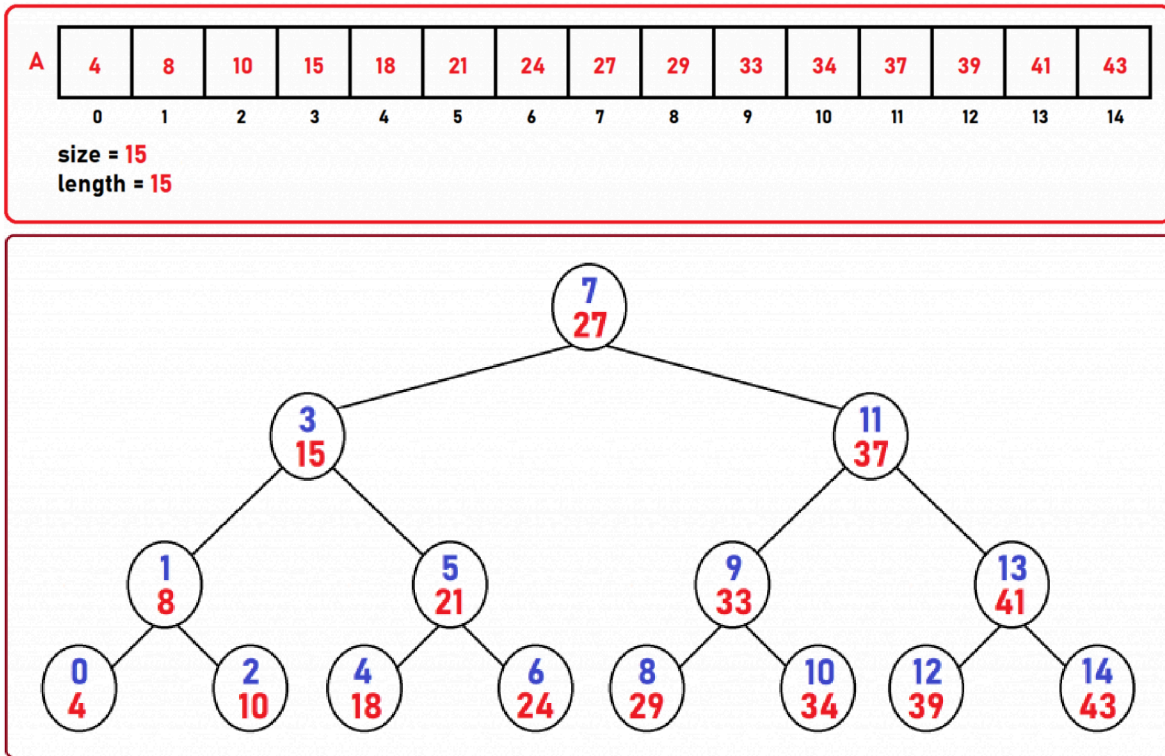
$$k = \lfloor \log_2 n \rfloor + 1.$$

8. Therefore, the number of comparisons in the worst case is $\lfloor \log_2 n \rfloor + 1$, which is $O(\log n)$.

Average Time Complexity

The average case of Binary Search occurs when the target element is found after a number of comparisons proportional to the height of the comparison tree. On average, the search requires approximately $\log_2 n$ comparisons. Therefore, the average case time complexity of Binary Search is

$$O(\log n)$$



Step 1: Represent Binary Search as a Comparison Tree

In Binary Search, each comparison divides the search space into two halves. This process can be visualized as a binary comparison tree where:

- The root (level 0) represents the first comparison (middle element).
- Each level i contains 2^i elements.
- An element at level i requires $(i + 1)$ comparisons to be found.

Table 3.1: Binary Search Comparison Tree Structure

Level i	Number of elements at this level	Comparisons per element
0	$2^0 = 1$	1
1	$2^1 = 2$	2
2	$2^2 = 4$	3
3	$2^3 = 8$	4
$\vdots$	$\vdots$	$\vdots$
k	2^k	$k + 1$

Step 2: Compute the Total Number of Comparisons

The total number of comparisons T is obtained by multiplying the number of elements at each level by the comparisons required for that level, and summing across all levels:

$$T(\text{Total comparisons for searching all elements once}) = (1 \times 1) + (2 \times 2) + (4 \times 3) + (8 \times 4) + \cdots + (2^k \times (k + 1))$$

$$T = \sum_{i=0}^k (i + 1) \cdot 2^i$$

Splitting the sum:

$$T = \sum_{i=0}^k i \cdot 2^i + \sum_{i=0}^k 2^i$$

Using the standard identities:

$$\sum_{i=0}^k i \cdot 2^i = (k-1) \cdot 2^{k+1} + 2 \quad \text{and} \quad \sum_{i=0}^k 2^i = 2^{k+1} - 1$$

We obtain:

$$T = (k-1) \cdot 2^{k+1} + 2 + 2^{k+1} - 1 = k \cdot 2^{k+1} + 1$$

Step 3: Compute the Total Number of Elements

The total number of elements in a perfectly balanced comparison tree of height k is:

$$n = 1 + 2 + 4 + 8 + \dots + 2^k = \sum_{i=0}^k 2^i = 2^{k+1} - 1$$

Step 4: Compute the Average Number of Comparisons

The average number of comparisons is the total comparisons divided by the total number of elements:

$$\text{Avg} = \frac{T}{n} = \frac{k \cdot 2^{k+1} + 1}{2^{k+1} - 1}$$

Step 5: Express the Result in Terms of n

From $n = 2^{k+1} - 1$, we have:

$$2^{k+1} = n + 1 \implies k = \log_2(n + 1) - 1$$

Substituting these into the expression for Avg:

$$\begin{aligned} \text{Avg} &= \frac{(\log_2(n + 1) - 1)(n + 1) + 1}{n} \\ &= \frac{(n + 1) \log_2(n + 1) - (n + 1) + 1}{n} \\ &= \frac{(n + 1) \log_2(n + 1) - n}{n} \\ &= \frac{n + 1}{n} \log_2(n + 1) - 1 \end{aligned}$$

Step 6: Apply Asymptotic Approximation

For large n :

$$\frac{n + 1}{n} \approx 1 \quad \text{and} \quad \log_2(n + 1) \approx \log_2 n$$

Therefore:

$$\text{Avg} \approx \log_2(n + 1) - 1 \approx \log_2 n - 1$$

Step 7: Conclusion

Since the dominant term in the average number of comparisons is $\log_2 n$, the average case time complexity of Binary Search is:

$$O(\log n)$$

Note:-

The above derivation assumes:

1. **Complete comparison tree** — The Binary Search process is represented as a perfectly balanced binary comparison tree, where every internal level is completely filled.
2. **Uniform probability** — All elements are equally likely to be searched, so each leaf of the comparison tree contributes with the same weight when computing the average.
3. **Specific array size** — The array size follows the form

$$n = 2^{k+1} - 1$$

so that the tree has exactly $k + 1$ fully-filled levels and remains perfectly balanced.

3.2 Max-Min Finding

Definition 3.2.1

Given an array A of size n , we want to find both the maximum and the minimum element simultaneously.

3.2.1 Conventional Approach (Linear Scan)

In the traditional approach, the maximum and minimum element can be found by comparing each element and updating the Max and Min values as and when required.

Algorithm 3.2.1

Input: Array A of size n

Output: Maximum (Max) and minimum (Min) elements

1. Set $\text{Max} \leftarrow A[0]$, $\text{Min} \leftarrow A[0]$
2. For $i \leftarrow 1$ to $n - 1$ do:
 - (a) If $A[i] > \text{Max}$ then $\text{Max} \leftarrow A[i]$
 - (b) If $A[i] < \text{Min}$ then $\text{Min} \leftarrow A[i]$
3. Return Max, Min

3.2.2 Analysis of Conventional Approach

This approach does $(n - 1)$ comparisons for finding the maximum and the same number for finding the minimum. The total number of comparisons is:

$$2(n - 1) = 2n - 2.$$

3.2.3 Divide and Conquer Approach

Using divide and conquer, we can reduce the number of comparisons.

Algorithm 3.2.2**Input:** Array A , left index l , right index r **Output:** Maximum and minimum of subarray $A[l..r]$

1. If $l = r$ then:
 - (a) $\max \leftarrow A[l]$, $\min \leftarrow A[l]$
2. Else if $r = l + 1$ then:
 - (a) If $A[l] > A[r]$ then:
 - i. $\max \leftarrow A[l]$, $\min \leftarrow A[r]$
 - (b) Else:
 - i. $\max \leftarrow A[r]$, $\min \leftarrow A[l]$
3. Else:
 - (a) $\text{mid} \leftarrow \lfloor (l + r) / 2 \rfloor$
 - (b) $(\max1, \min1) \leftarrow \text{MaxMin}(A, l, \text{mid})$
 - (c) $(\max2, \min2) \leftarrow \text{MaxMin}(A, \text{mid} + 1, r)$
 - (d) If $\max1 > \max2$ then $\max \leftarrow \max1$ else $\max \leftarrow \max2$
 - (e) If $\min1 < \min2$ then $\min \leftarrow \min1$ else $\min \leftarrow \min2$
4. Return $(\max, \min)$

3.2.4 Recurrence Relation for Divide and Conquer Approach

Let $T(n)$ be the number of comparisons required for an array of size n . Then:

$$T(n) = \begin{cases} 0, & n = 1 \\ 1, & n = 2 \\ 2T\left(\frac{n}{2}\right) + 2, & n > 2 \end{cases}$$

Solving the Recurrence

We solve the recurrence using the iterative (substitution) method.

1. Start with $T(n) = 2T(n/2) + 2$ for $n > 2$.
2. Substitute $T(n/2) = 2T(n/4) + 2$:

$$T(n) = 2[2T(n/4) + 2] + 2 = 4T(n/4) + 6.$$

3. Substitute $T(n/4) = 2T(n/8) + 2$:

$$T(n) = 8T(n/8) + 14.$$

$$T(n) = 8T(n/8) + 8 + 4 + 2.$$

$$T(n) = 2^3T(n/2^3) + 2^3 + 2^2 + 2^1.$$

4. After k substitutions:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + (2^{k+1} - 2).$$

5. Stop when $\frac{n}{2^k} = 2$ (base case), so $n = 2^{k+1}$ and $k = \log_2 n - 1$, $2^k = n/2$.
6. Since $T(2) = 1$:

$$T(n) = \frac{n}{2} \cdot 1 + n - 2 = \frac{3n}{2} - 2.$$

7. Therefore:

$$T(n) = \frac{3n}{2} - 2.$$

3.2.5 Comparison of Approaches

Approach	Number of Comparisons
Conventional (linear scan)	$2n - 2$
Divide and Conquer	$\frac{3n}{2} - 2$

Note:-

For large n , the divide and conquer approach saves approximately $\frac{n}{2}$ comparisons (a 25% reduction). However, it always performs exactly $\frac{3n}{2} - 2$ comparisons, regardless of input order.

3.2.6 Time Complexity

Hence, both approaches have linear time complexity:

$$T(n) = O(n).$$

Example 3.2.1

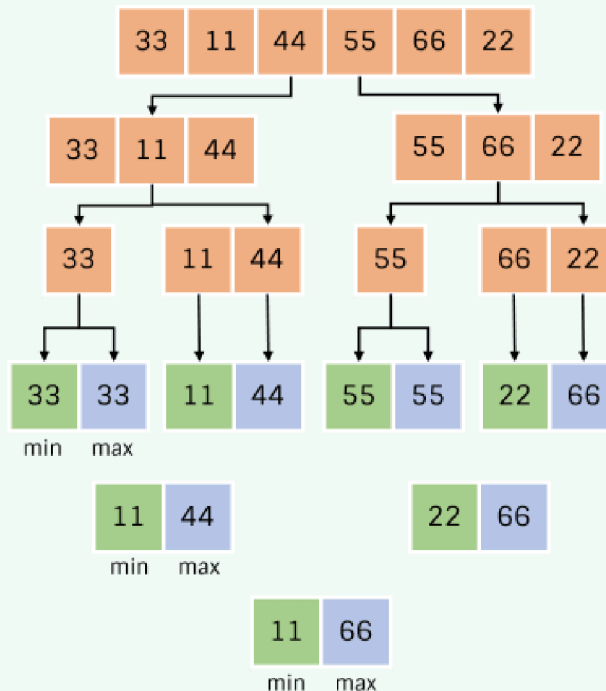
Find max and min from the sequence 33, 11, 44, 55, 66, 22 using divide and conquer approach.

Solution:

We apply the divide and conquer Max-Min algorithm to the array:

$$A = [33, 11, 44, 55, 66, 22]$$

The array is recursively divided into halves until each subarray contains either one or two elements. Then, the maximum and minimum of each pair are combined as we conquer back up the recursion tree.



Note: The orange cell represents the stage of stepwise division. In the conquer stage, the smallest element from the parent arrays is placed in the green cell, while the largest element is stored in the blue cell.

Result:

$$\text{Max} = 66, \quad \text{Min} = 11$$

Number of comparisons:

Using the formula $T(n) = \frac{3n}{2} - 2$ for $n = 6$:

$$T(6) = \frac{3 \cdot 6}{2} - 2 = 9 - 2 = 7 \text{ comparisons}$$

Compared to the conventional approach which would take $2n - 2 = 10$ comparisons, the divide and conquer approach saves 3 comparisons.

3.2.7 Space Complexity

Conventional (Iterative) Approach

The conventional algorithm uses only a constant number of variables:

- Max and Min to store the current maximum and minimum.
- Loop index i .
- No additional arrays or recursion.

Therefore:

$$\text{Space Complexity} = O(1)$$

Divide and Conquer Approach

The recursive divide-and-conquer algorithm uses additional memory due to recursion:

- Each recursive call adds a stack frame (storing parameters l , r , and return addresses).
- The maximum depth of recursion is $\log_2 n$ (since the array is repeatedly halved).
- No extra arrays are created – the merging step uses only comparisons, not temporary storage.

Hence:

$$\text{Space Complexity} = O(\log n)$$

Comparison

Approach	Space Complexity
Conventional (iterative)	$O(1)$
Divide and Conquer	$O(\log n)$

Note:-

The divide and conquer version uses $O(\log n)$ stack space, which is generally acceptable even for large n . However, for extremely deep recursion (e.g., $n > 10^6$), the stack limit may be reached in some programming languages. An iterative version avoids this issue.

3.3 Merge Sort

Merge sort sorts a list using the divide-and-conquer strategy. Unlike selection sort, bubble sort, or insertion sort, it sorts data by dividing the list into sublists, recursively solving them, and then combining the results.

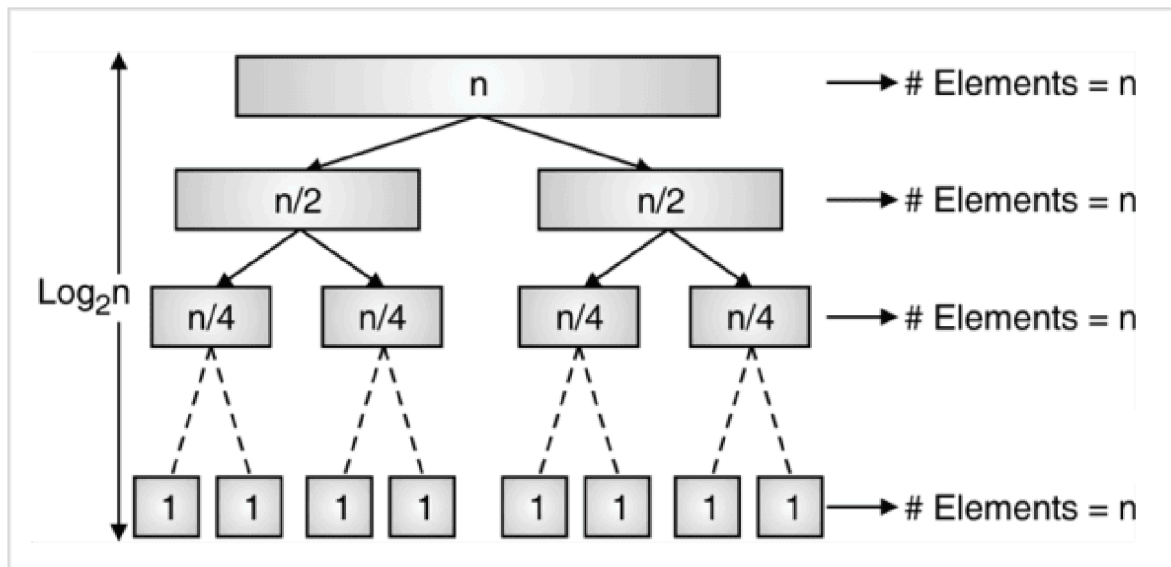
It works on the following two basic principles:

1. Sorting a smaller list takes less time than sorting a larger list.
2. Combining two sorted sublists takes less time than combining two unsorted lists.

3.3.1 Three Stages of Merge Sort

1. **Divide:** Recursively divide the single list into two sublists until each sublist contains exactly one element.
2. **Conquer:** Sort both sublists of the parent list. A list containing only one element is trivially sorted.
3. **Combine:** Recursively merge the sorted sublists until a sorted list of size n is produced.

Merge sort divides the list of size n into two sublists, each of size $n/2$. This subdivision continues until the problem size becomes one. After hitting the problem size of one, the conquer phase begins. The following figure shows the divide phase.



3.3.2 Algorithm

Algorithm 3.3.1

MergeSort(A, l, r)

Input: Array A , left index l , right index r

Output: Sorted array A

1. **if** $l < r$ **then**
 - (a) $mid \leftarrow (l + r) / 2$
 - (b) MergeSort(A, l, mid) // Sort left half
 - (c) MergeSort($A, mid + 1, r$) // Sort right half
 - (d) Merge(A, l, mid, r) // Merge the two sorted halves
2. **end if**

Algorithm 3.3.2**Merge(A, l, mid, r)****Input:** Array A , left index l , middle index mid , right index r **Output:** Merged sorted subarray within $A[l \dots r]$

```

1.  $n_1 \leftarrow mid - l + 1$ 
2.  $n_2 \leftarrow r - mid$ 
3. Create arrays  $L[0 \dots n_1 - 1]$  and  $R[0 \dots n_2 - 1]$ 
4. for  $i \leftarrow 0$  to  $n_1 - 1$  do
    (a)  $L[i] \leftarrow A[l + i]$ 
5. end for
6. for  $j \leftarrow 0$  to  $n_2 - 1$  do
    (a)  $R[j] \leftarrow A[mid + 1 + j]$ 
7. end for
8.  $i \leftarrow 0, j \leftarrow 0, k \leftarrow l$ 
9. while  $i < n_1$  and  $j < n_2$  do
    (a) if  $L[i] \leq R[j]$  then
        i.  $A[k] \leftarrow L[i]$ 
        ii.  $i \leftarrow i + 1$ 
    (b) else
        i.  $A[k] \leftarrow R[j]$ 
        ii.  $j \leftarrow j + 1$ 
    (c) end if
    (d)  $k \leftarrow k + 1$ 
10. end while
11. while  $i < n_1$  do                                     // Copy remaining elements of L
    (a)  $A[k] \leftarrow L[i]$ 
    (b)  $i \leftarrow i + 1$ 
    (c)  $k \leftarrow k + 1$ 
12. end while
13. while  $j < n_2$  do                                     // Copy remaining elements of R
    (a)  $A[k] \leftarrow R[j]$ 
    (b)  $j \leftarrow j + 1$ 
    (c)  $k \leftarrow k + 1$ 
14. end while

```

Example 3.3.1

Simulate merge sort on the data sequence: 77, 22, 33, 44, 11, 55, 66.

Solution:

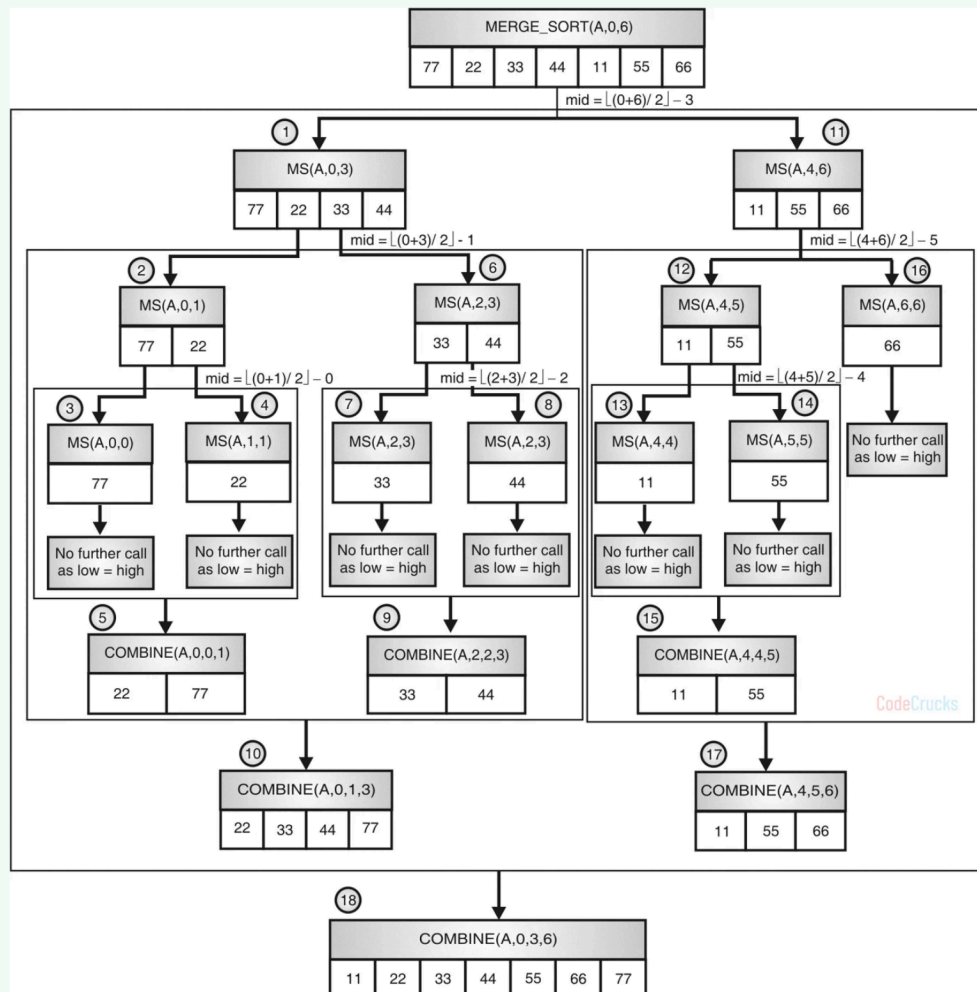
Consider the input array:

$$A = [77, 22, 33, 44, 11, 55, 66]$$

77	22	33	44	11	55	66
0	1	2	3	4	5	6
↑						↑
low						high

The simulation of merge sort for this array is shown in the figure below. The number in a circle on the left or right corner indicates the call sequence. COMBINE is called as soon as two sublists of size one are produced.

Step-by-step trace:



Final sorted list:

{11, 22, 33, 44, 55, 66, 77}

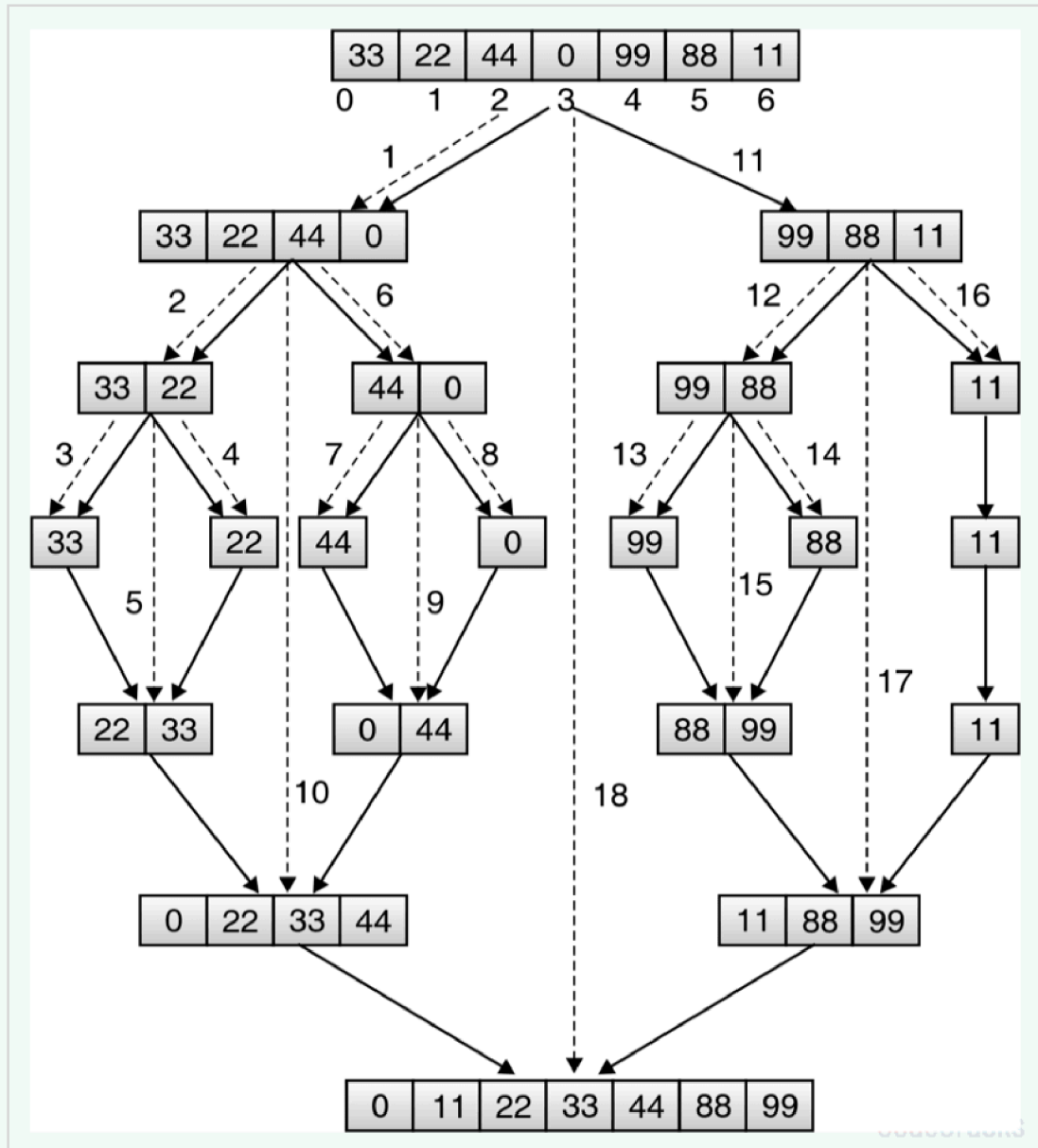
Example 3.3.2

Sort the sequence using merge sort: $A = \{33, 22, 44, 00, 99, 88, 11\}$

Solution:

Merge sort proceeds by repeatedly splitting the array around the midpoint until each piece holds a single element, then merging back upward.

The variables *low* and *high* represent the first and last index of the passed array. The call sequence is shown in the figure above.



Final sorted list:

$\{00, 11, 22, 33, 44, 88, 99\}$

Question 3.1: Assignment

Sort the following elements using merge sort:

410, 385, 279, 752, 451, 523, 961, 354, 550, 620

3.3.3 Analysis of Merge Sort

The total running time of merge sort is computed by summing the complexity of its three phases:

1. Divide:

In merge sort, we do *not* physically split the array. The array of size n is divided into two subarrays by computing the middle index (one addition and one division). This operation takes constant time:

$$D(n) = O(1)$$

2. Conquer:

We recursively solve two subproblems, each of size $n/2$. This contributes $2T(n/2)$ to the running time.

3. Combine:

The COMBINE (merge) procedure merges two sublists, each of size $n/2$, and performs n comparisons in the worst case:

$$C(n) = O(n)$$

Recurrence relation:

$$T(n) = \begin{cases} O(1), & \text{if } n = 1 \\ 2T(n/2) + O(n), & \text{if } n > 1 \end{cases}$$

We derive the time complexity of merge sort using two methods:

1. Substitution method
2. Master method

Master Method

The recurrence $T(n) = 2T(n/2) + n$ is of the form:

$$T(n) = a \cdot T(n/b) + f(n)$$

Comparing both:

$$a = 2, \quad b = 2, \quad f(n) = n$$

Compute $n^{\log_b a}$:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Since $f(n) = n = \Theta(n^{\log_b a})$, this matches **Case 3** of the Master Theorem:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n \log n)$$

$$T(n) = O(n \log n)$$

Note on All Three Cases

Whether the list is already sorted, inverse sorted, or randomly shuffled, all three phases (divide, conquer, combine) must still be performed. Merge sort *cannot detect* if the list is already sorted, so the number of comparisons remains the same in all cases. The time complexity of merge sort is therefore identical for best, average, and worst cases.

Table 3.2: Time Complexity of Merge Sort

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

3.3.4 Space Complexity

Merge sort requires additional memory beyond the input array because of the auxiliary arrays used during the merge step.

Analysis:

In each call to the Merge procedure, two temporary arrays L and R are created to hold the elements of the two halves being merged:

$$\text{Size of } L = n_1 = \text{mid} - l + 1$$

$$\text{Size of } R = n_2 = r - \text{mid}$$

Together, these auxiliary arrays hold all n elements of the current subarray. Therefore, the merge step requires $O(n)$ extra space.

Additionally, the recursive calls use the call stack, which can grow up to a depth of $\log_2 n$ levels. This contributes $O(\log n)$ space for the recursion stack.

Total space complexity:

$$S(n) = O(n) + O(\log n) = O(n)$$

$S(n) = O(n)$

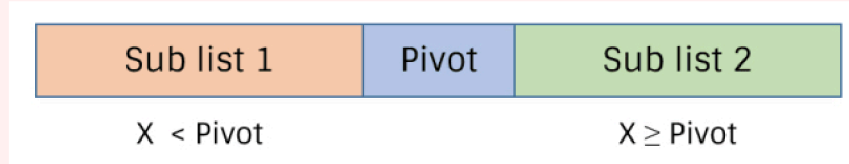
Note:-

Unlike in-place sorting algorithms such as quick sort or heap sort, merge sort is *not* an in-place algorithm because it requires $O(n)$ auxiliary space. This is one of its main drawbacks when memory is a constraint.

3.4 Quick Sort

Definition 3.4.1

Quick sort was invented by Tony Hoare. It is a divide-and-conquer strategy. Quicksort is sometimes referred to as **partition exchange sort**. Quick sort picks one element as a **pivot** and shifts the pivot to its correct location in each iteration. By repositioning the pivot, the list is divided into two sublists (possibly unequal). Each sublist is solved recursively.



In contrast to merge sort, the list is partitioned dynamically based on the **actual values** (not positions). Merge sort splits from the centre; quicksort's division depends on the pivot value. The pivot can be the first, last, random, or median element.

3.4.1 Algorithm

QuickSort (Main Routine)

Algorithm 3.4.1

Input: Array A , starting index low , ending index $high$ & **Output:** Sorted array A

1. If $low < high$ then:
 - (a) $p \leftarrow \text{Partition}(A, low, high)$
 - (b) $\text{QuickSort}(A, low, p - 1)$
 - (c) $\text{QuickSort}(A, p + 1, high)$
2. End if

Partition (using first element as pivot)

Algorithm 3.4.2

Input: Array A , low , $high$ & **Output:** New pivot index y

1. $pivot \leftarrow A[low]$, $x \leftarrow low + 1$, $y \leftarrow high$
2. While $x \leq y$ do:
 - (a) While $x \leq high$ and $A[x] \leq pivot$ do:
 - i. $x \leftarrow x + 1$
 - (b) While $y \geq low + 1$ and $A[y] > pivot$ do:
 - i. $y \leftarrow y - 1$
 - (c) If $x < y$ then swap $A[x]$ and $A[y]$
3. Swap $A[low]$ with $A[y]$ (place pivot in correct position)
4. Return y

3.4.2 Pseudocode (Python)

Quick Sort Implementation

```

1  def partition(arr, low, high):
2      pivot = arr[low]
3      x = low + 1
4      y = high
5      while x <= y:
6          while x <= high and arr[x] <= pivot:
7              x += 1
8          while y >= low+1 and arr[y] > pivot:
9              y -= 1
10         if x < y:
11             arr[x], arr[y] = arr[y], arr[x]
12     arr[low], arr[y] = arr[y], arr[low]
13     return y
14
15 def quick_sort(arr, low, high):
16     if low < high:
17         p = partition(arr, low, high)
18         quick_sort(arr, low, p-1)
19         quick_sort(arr, p+1, high)

```

3.4.3 Example (Tracing Quick Sort)

Example 3.4.1 (Trace quick-sort for the data set $A = \{44, 22, 33, 77, 11, 55, 66\}$)

Step 1: Initial array

Initially,

44	22	33	77	11	55	66
0	1	2	3	4	5	6

Step 2: First partition (low=0, high=6)

pivot

44	22	33	77	11	55	66
0	1	2	3	4	5	6

low high

$$pivot = A[0] = 44, \quad x = low + 1 = 0 + 1 = 1, \quad y = high = 6$$

While loop (Algorithm line: while $x \leq y$ do)

- **Iteration 1:** $x = 1, y = 6$, condition $1 \leq 6$ true.

– **Move x right (Algorithm: while $x \leq high$ and $A[x] \leq pivot$):**

$$A[1] = 22 \leq 44 \Rightarrow x = 2; \quad A[2] = 33 \leq 44 \Rightarrow x = 3; \quad A[3] = 77 \leq 44 \text{ false.}$$

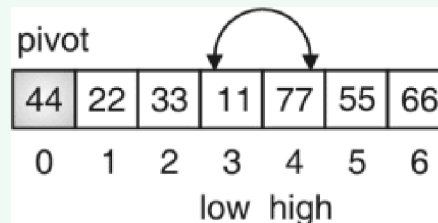
Stop with $x = 3$.

- **Move y left (Algorithm: while $y \geq low + 1$ and $A[y] > pivot$):**

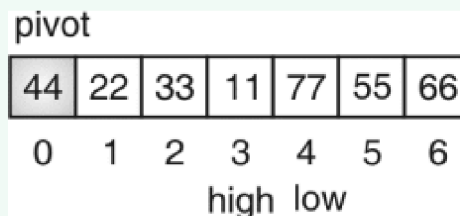
$A[6] = 66 > 44 \Rightarrow y = 5$; $A[5] = 55 > 44 \Rightarrow y = 4$; $A[4] = 11 > 44$ false.

Stop with $y = 4$.

- **Check and swap (Algorithm: if $x < y$ then swap $A[x]$ with $A[y]$):** Since $3 < 4$, swap $A[3]$ and $A[4]$.

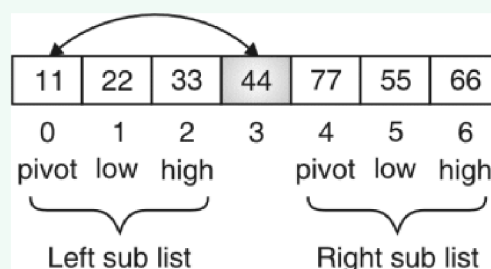


- **Iteration 2:** Continue while loop: now $x = 3$, $y = 4$, condition $3 \leq 4$ true.
 - Move x : $A[3] = 11 \leq 44 \Rightarrow x = 4$; $A[4] = 77 \leq 44$ false. $x = 4$.
 - Move y : $A[4] = 77 > 44 \Rightarrow y = 3$; now $y = 3$.
 - Check $x < y$: $4 < 3$ false, no swap.
- Loop condition re-evaluated: $x = 4$, $y = 3$, $4 \leq 3$ false, exit while.



Place pivot (Algorithm: swap $A[low]$ with $A[y]$): Swap $A[0]$ and $A[3]$ (since $y = 3$):

$[44, 22, 33, 11, 77, 55, 66] \rightarrow [11, 22, 33, 44, 77, 55, 66]$



Return pivot index $y = 3$.

Recursive calls (Algorithm: QuickSort(A , low, $p-1$) and QuickSort(A , $p+1$, high)):

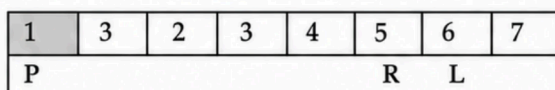
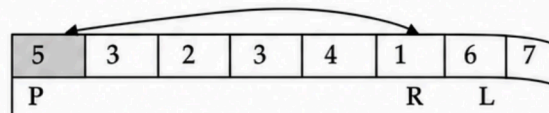
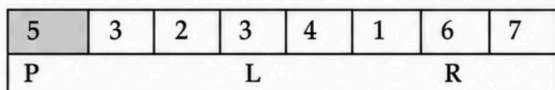
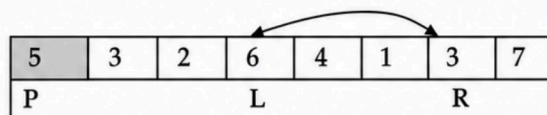
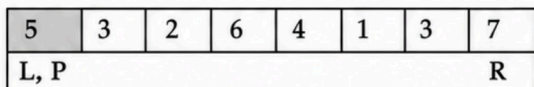
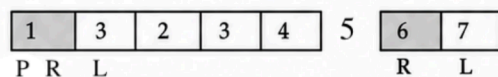
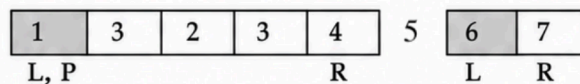
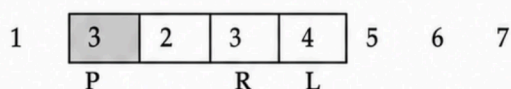
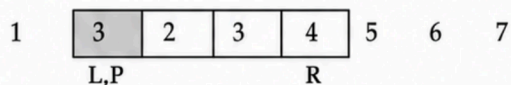
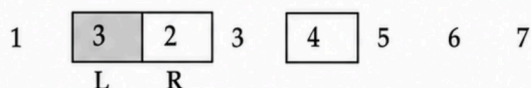
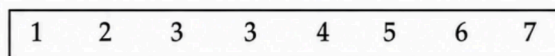
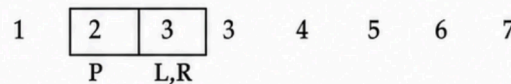
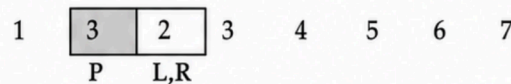
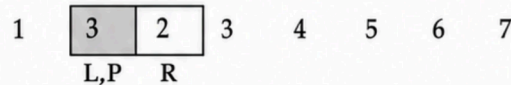
- Left subarray: $[11, 22, 33]$ (indices 0–2)
- Right subarray: $[77, 55, 66]$ (indices 4–6)

The same partition logic is applied to each subarray recursively.



Example 3.4.2

Initial array: [44, 22, 33, 77, 11, 55, 66].

Tracing: Sort the following data items by using Quick sort $A[] = \{5, 3, 2, 6, 4, 1, 3, 7\}$ **Pass 1:****Pass 2:****Pass 3:****Pass 4:****Pass 5:****3.4.4 Analysis of Quick Sort****Best Case Complexity**

Best case occurs when the pivot always divides the array into two nearly equal halves. This gives a recurrence identical to merge sort:

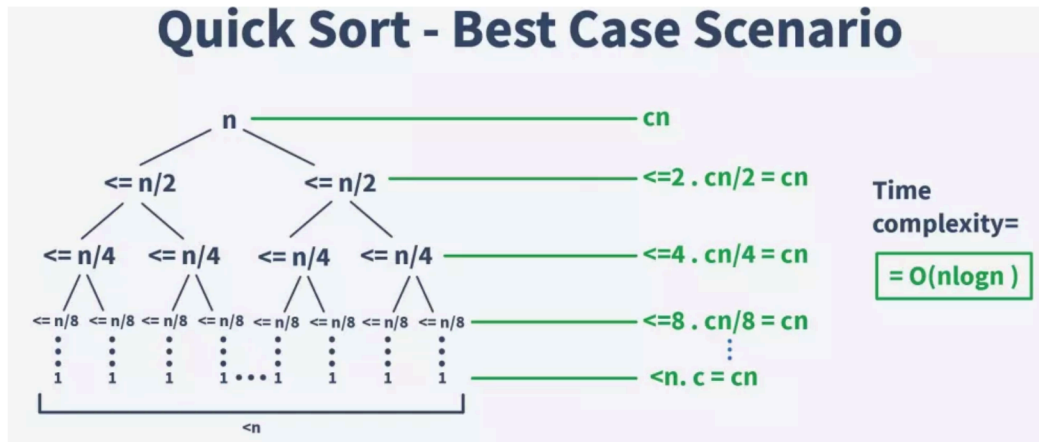
$$T(n) = 2T(n/2) + O(n) + O(1)$$

Ignoring the constant term:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

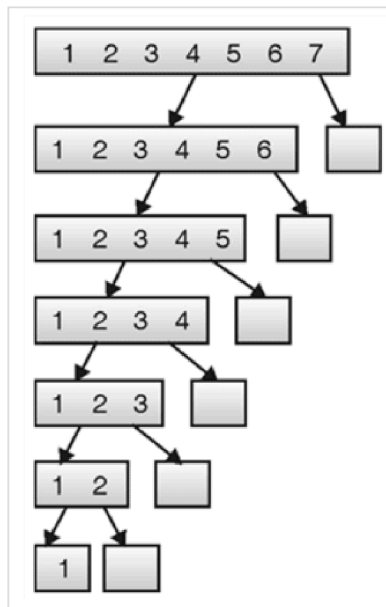
Solving yields:

$$T_{\text{best}}(n) = O(n \log n)$$



Worst Case Complexity

The worst case occurs when the pivot is always the smallest or largest element (e.g., already sorted array). One subproblem has size $n - 1$ and the other size 0.



- **Divide cost:** $O(n)$ – we must compare each element to the pivot and identify the right location of the pivot to split the list.
- **Conquer cost:** $T(n - 1)$ – one subproblem has size $n - 1$, the other is empty. Thus the algorithm must recursively solve a problem of size $n - 1$ in the next call. Hence the conquer cost is $T(n - 1)$.
- **Combine cost:** $O(1)$ – the pivot is already in its correct position. Because each call sets the pivot in its correct final position and no further work is required, the combination cost for quicksort is $O(1)$.

The recurrence relation is:

$$T(n) = T(n-1) + O(n) + O(1), \quad T(0) = 1$$

Ignoring the constant term:

$$T(n) = T(n-1) + n, \quad T(0) = 1$$

Solving the recurrence (iterative method): We have the recurrence:

$$T(n) = T(n-1) + n \tag{1}$$

Using iterative approach, substitute n by $(n-1)$ in Equation (1):

$$T(n-1) = T(n-2) + (n-1) \tag{2}$$

$$\Rightarrow T(n) = T(n-2) + (n-1) + n \tag{3}$$

Substitute n by $(n-1)$ in Equation (2):

$$T(n-2) = T(n-3) + (n-2)$$

From Equation (3):

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

$$\vdots \quad \vdots$$

After k iterations:

$$T(n) = T(n-k) + (n-k+1) + (n-k+2) + \cdots + (n-1) + n$$

Let $k = n$:

$$T(n) = T(0) + 1 + 2 + 3 + \cdots + (n-2) + (n-1) + n$$

$$T(n) = 1 + 1 + 2 + 3 + \cdots + n$$

Ignoring constants and lower-order terms:

$$T_{\text{worst}}(n) = O(n^2)$$

Note:-

This is why the naive version of Quick Sort (always picking the first or last element as pivot) performs poorly on already sorted data. Randomised Quick Sort or median-of-three pivot selection avoids this worst case with high probability.

Average Case Complexity

On average, the pivot splits the array such that each subproblem size is roughly $n/2$. Therefore: The recurrence for the average case is:

$$T(n) = T(\text{divide}) + T(\text{conquer}) + T(\text{combine})$$

We know:

- $T(\text{divide}) = O(n)$ – comparing each element to the pivot.
- $T(\text{conquer}) = T(k) + T(n-k-1)$, where k is the size of the left subarray.
- $T(\text{combine}) = O(1)$ – no merging needed.

Thus,

$$T(n) = O(n) + T(k) + T(n-k-1) + O(1)$$

Ignoring constant terms,

$$T(n) = T(k) + T(n-k-1) + n$$

On average, the pivot ends up anywhere in the array, meaning the average size of the left and right subarrays is roughly $n/2$. Therefore, we approximate:

$$T(n) \approx 2T\left(\frac{n}{2}\right) + n$$

Solving this recurrence using the Master Theorem Hence,

$$T_{\text{avg}}(n) = O(n \log n)$$

Note:-

The average case of Quick Sort is $O(n \log n)$, which makes it one of the fastest sorting algorithms in practice, despite the worst-case $O(n^2)$.

3.4.5 Space Complexity

Only two components use extra memory:

1. **Recursion stack:** Each call stores parameters (low, high) and return address. This is the dominant factor, depending on input order.
2. **Constants:** A few local variables (pivot, x, y, temp) – these are $O(1)$ regardless of n .

$$S(n) = O(\text{recursion depth}) + O(1) = O(\text{recursion depth})$$

- **Best and average case:** The pivot divides the array into two balanced halves. The recursion depth is the height of a balanced binary tree, which is $\log_2 n$. Thus,

$$S_{\text{avg}}(n) = O(\log n)$$

Reason: Balanced partitions $\rightarrow$ each recursive call reduces the problem size by half $\rightarrow$ depth $\log n$.

- **Worst case (array already sorted or reverse sorted):** The pivot is always the smallest or largest element. Each recursive call handles a subarray of size $n - 1$, while the other side is empty. The recursion degenerates into a chain of n calls. Thus,

$$S_{\text{worst}}(n) = O(n)$$

Reason: Unbalanced partitions $\rightarrow$ only one subproblem shrinks by 1 each time $\rightarrow$ depth n .

All other local variables use $O(1)$ space and do not affect the asymptotic complexity.

$$S(n) = \begin{cases} O(\log n) & \text{best/average case} \\ O(n) & \text{worst case} \end{cases}$$

3.4.6 Comparison of Pivot Selection Strategies

Strategy	Effect
First/Last element	Worst case on already sorted input
Random element	Avoids worst case with high probability
Median-of-three	More robust, balanced partitions

Question 3.2

1. What is the worst case of quick sort and how does randomized quick sort handle this problem?
2. Sort the data $\{-2, 4, -3, 6, 12, 10, 11, 13, 9\}$ using quick sort (show steps).

Note:-

Randomised quick sort picks a random pivot, making the worst-case input very unlikely. The expected time remains $O(n \log n)$.

3.5 Randomized Quick Sort

Definition 3.5.1: Randomized Quick Sort

Randomized Quick Sort is a variation of the standard quicksort algorithm in which the pivot element is chosen **randomly** rather than deterministically.

- Instead of always picking a fixed position (like the first or last element) as the pivot, the algorithm randomly selects an element from the array. This randomization reduces the likelihood of consistently poor pivot choices.
- Random pivot selection helps avoid worst-case scenarios (already sorted or reverse sorted arrays).
- It achieves expected running time of $O(n \log n)$ for all input sequences.
- Significantly reduces the probability of encountering the worst-case scenario $O(n^2)$ compared to deterministic quicksort.
- Balances partition sizes on average, leading to a more balanced recursion tree.
- Provides more consistent performance across different data distributions (uniform, normal, skewed).

3.5.1 Algorithm (Step-by-Step)

RandomizedQuickSort

Algorithm 3.5.1

Input: Array A , starting index low , ending index $high$

Output: Sorted array A

1. If $low < high$ then:
 - (a) $p \leftarrow \text{RandomizedPartition}(A, low, high)$
 - (b) $\text{RandomizedQuickSort}(A, low, p - 1)$
 - (c) $\text{RandomizedQuickSort}(A, p + 1, high)$
2. End if

RandomizedPartition

Algorithm 3.5.2

Input: Array A , low , $high$

Output: New pivot index y

1. $r \leftarrow \text{Random}(low, high)$ (choose random index)
2. Swap $A[r]$ with $A[low]$ (move random element to first position)
3. $pivot \leftarrow A[low]$ (pivot = first element)
4. $x \leftarrow low + 1$
5. $y \leftarrow high$
6. While $x \leq y$ do:
 - (a) While $x \leq high$ and $A[x] \leq pivot$ do $x \leftarrow x + 1$
 - (b) While $y \geq low + 1$ and $A[y] > pivot$ do $y \leftarrow y - 1$
 - (c) If $x < y$ then swap $A[x]$ and $A[y]$
7. Swap $A[low]$ with $A[y]$ (place pivot in correct position)
8. Return y

3.5.2 Pseudocode

Randomized Quick Sort

```

1 def randomized_quick_sort(A, low, high):
2     if low < high:
3         p = randomized_partition(A, low, high)
4         randomized_quick_sort(A, low, p - 1)
5         randomized_quick_sort(A, p + 1, high)
6
7 def randomized_partition(A, low, high):
8     r = random.randint(low, high) # choose random index
9     A[r], A[low] = A[low], A[r] # swap with first element
10    pivot = A[low]
11    x = low + 1
12    y = high
13    while x <= y:
14        while x <= high and A[x] <= pivot:
15            x += 1
16        while y >= low + 1 and A[y] > pivot:
17            y -= 1
18        if x < y:
19            A[x], A[y] = A[y], A[x]
20    A[low], A[y] = A[y], A[low] # place pivot in correct position
21    return y

```

3.5.3 Analysis

Randomized Quick Sort achieves expected $O(n \log n)$ time for all input distributions by randomizing pivot selection, significantly reducing the chance of worst-case behavior.

- Even if the array is already sorted or reverse sorted, the pivot is chosen randomly, so the input order does not affect performance.
- Although the worst case $O(n^2)$ is not completely eliminated, its probability is extremely low and depends only on random choices rather than the input itself.

Hence, randomized quick sort provides more reliable and consistent performance in practice.

3.5.4 Expected Time Complexity

The recurrence for the expected number of comparisons is:

$$T(n) = \frac{1}{n} \sum_{k=0}^{n-1} (T(k) + T(n-k-1)) + O(n)$$

Solving this recurrence gives:

$$E[T(n)] = O(n \log n)$$

3.5.5 Space Complexity

The space complexity is the same as deterministic Quick Sort (recursion stack):

$$S(n) = O(\log n) \quad \text{expected auxiliary space.}$$

Note:-

The worst-case space $O(n)$ is still possible but with exponentially small probability. In practice, randomized quick sort behaves as if the worst case never occurs.

Question 3.3: Assignment

1. Explain how random pivot selection reduces the probability of worst-case partition.
2. Implement randomized quick sort in your preferred language and test it on a sorted array of size 1000. Compare its runtime with deterministic quick sort (first element pivot).

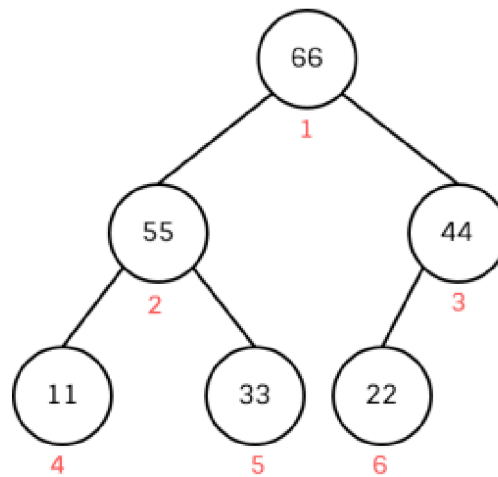
3.6 Heap Sort

Definition 3.6.1

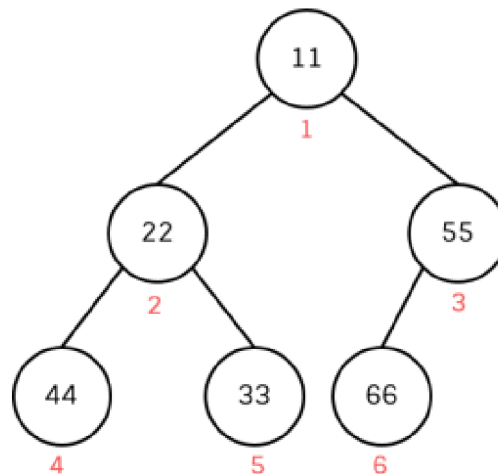
Heap Sort is a comparison-based sorting algorithm based on the Binary Heap data structure. It is an optimized version of selection sort. The algorithm uses a **Heap Tree** – a complete binary tree – and can employ either a max-heap or a min-heap.

3.6.1 Max Heap and Min Heap

- **Max heap:** The value of a parent node is greater than or equal to its children: $A[\text{PARENT}(i)] \geq A[i]$. The largest element is at the root. Max heap is used in heap sort (to produce ascending order).



- **Min heap:** The value of a parent node is less than or equal to its children: $A[\text{PARENT}(i)] \leq A[i]$. The smallest element is at the root. Min heap is used in priority queues.



3.6.2 Algorithm

Algorithm 3.6.1

HEAP_SORT(A)

Input: Array A of size n

Output: Sorted array A in ascending order

1. $n \leftarrow \text{length}(A)$
2. BUILD_MAX_HEAP(A, n) (build initial max heap)
3. For $i \leftarrow n$ down to 2 do:
 - (a) Swap $A[1]$ with $A[i]$ (move current maximum to end)
 - (b) $n \leftarrow i - 1$
 - (c) MAX_HEAPIFY($A, 1, n$)
4. Stop

Algorithm 3.6.2

BUILD_MAX_HEAP(A, n)

Input: Array A of size n

1. For $i \leftarrow \lfloor n/2 \rfloor$ down to 1 do: (floor value of $n/2$)
 - (a) MAX_HEAPIFY(A, i, n)
2. Stop

Algorithm 3.6.3

MAX_HEAPIFY(A, i, n)

Input: Array A , index i , heap size n

1. $L \leftarrow 2i$ (left child index, LEFT(i))
2. $R \leftarrow 2i + 1$ (right child index, RIGHT(i))
3. $largest \leftarrow i$
4. If $L \leq n$ and $A[L] > A[largest]$, then: $largest \leftarrow L$
5. If $R \leq n$ and $A[R] > A[largest]$, then: $largest \leftarrow R$
6. If $largest \neq i$, then:
 - (a) Swap $A[i]$ with $A[largest]$
 - (b) MAX_HEAPIFY($A, largest, n$)
7. Stop

3.6.3 Complexity Analysis

MAX_HEAPIFY

- Maintains the max-heap property at a given node by comparing with its children and moving down recursively.
- The height of a binary heap with n nodes is $\lfloor \log_2 n \rfloor$.
- In the worst case (e.g., smallest element at root), the algorithm travels from root to leaf, performing $O(\log n)$ swaps.

$$\text{MAX_HEAPIFY} = O(\log n)$$

BUILD_MAX_HEAP

- Calls MAX_HEAPIFY on roughly $n/2$ nodes, but the tighter analysis gives linear time because most heapify calls are on small sub-heaps.

$$\text{BUILD_MAX_HEAP} = O(n)$$

The key is this: The cost of MAX_HEAPIFY depends on the **height** of the node it's called on, not on the total number of elements in the heap.

In a binary heap, **height** means the distance from the node down to the deepest leaf (how many levels it can sink). The root has height roughly $\log n$, but most nodes are much lower in the tree. Leaves (height 0) cost almost nothing – just a comparison. Nodes just above leaves (height 1) do a small constant amount of work. Only nodes very close to the root have height close to $\log n$.

Now, BUILD_MAX_HEAP runs MAX_HEAPIFY on the bottom half of all nodes (about $n/2$ nodes). But these are mostly **low-height** nodes – because the majority of nodes are in the lower part of the tree. So they don't take $\log n$ time each.

When you compute the total work by summing over all nodes, you get: Many cheap calls (low height) + few expensive calls (high height) = total linear time. That's why the total is $O(n)$, not $O(n \log n)$, even though MAX_HEAPIFY on the root would cost $O(\log n)$.

HEAP_SORT

$$\text{BUILD_MAX_HEAP} : O(n)$$

For loop $i = n$ down to 2 : $n - 1$ iterations

$$\text{Swap } A[1] \text{ with } A[i] : O(1)$$

$$\text{MAX_HEAPIFY}(A, i) : O(\log n)$$

Hence, each iteration: $O(1) + O(\log n) = O(\log n)$.

$$\text{Total loop cost} : (n - 1) \cdot O(\log n) = O(n \log n)$$

$$\text{Overall} : O(n) + O(n \log n) = O(n \log n)$$

Thus,

$$T_{\text{heap}}(n) = O(n \log n)$$

Time Complexity in All Cases

Heap sort does not detect whether the array is already sorted; it performs the same number of operations regardless of input order. Hence:

$$T_{\text{best}}(n) = T_{\text{avg}}(n) = T_{\text{worst}}(n) = O(n \log n)$$

3.6.4 Space Complexity

Heap sort is an in place algorithm (uses only the input array plus a few extra variables). No recursion is used (MAX HEAPIFY can be implemented iteratively). Therefore:

$$S(n) = O(1)$$

Note:-

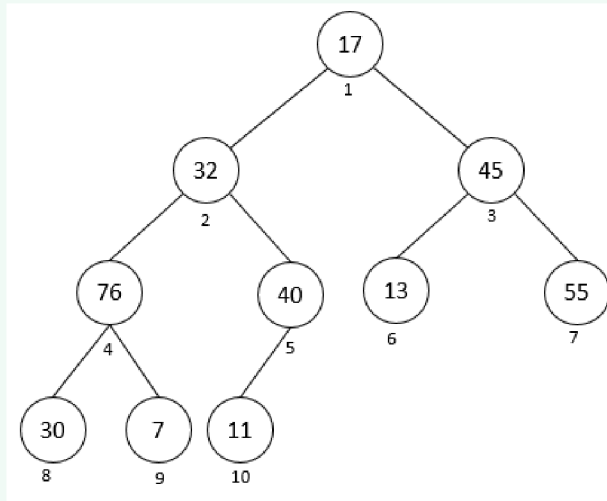
Unlike Quick Sort, Heap Sort guarantees $O(n \log n)$ even in the worst case, making it suitable for real-time systems. However, it is not stable and typically slower than well-tuned Quick Sort due to poor cache locality.

3.6.5 Detailed Example

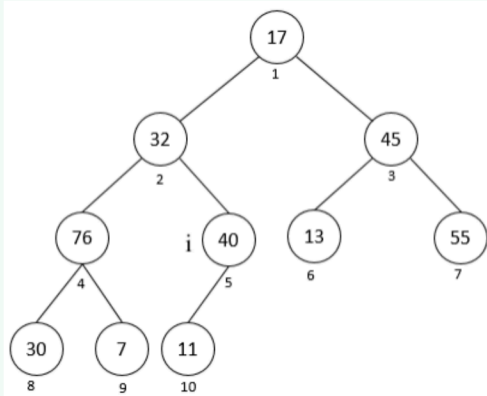
Example 3.6.1 (Sort $A = [17, 32, 45, 76, 40, 13, 55, 30, 7, 11]$ using Heap Sort)

Stage 1: Build Max Heap (initial heapification)

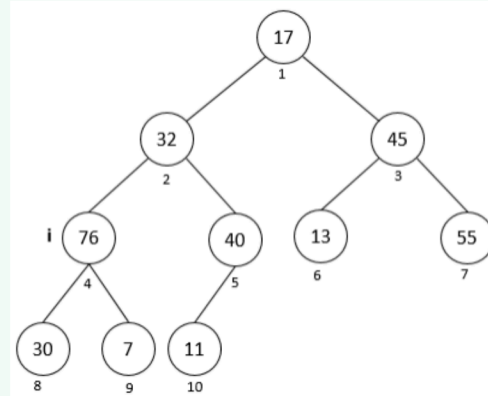
Here $n = 10$, start from $i = \lfloor n/2 \rfloor = 5$ down to 1.



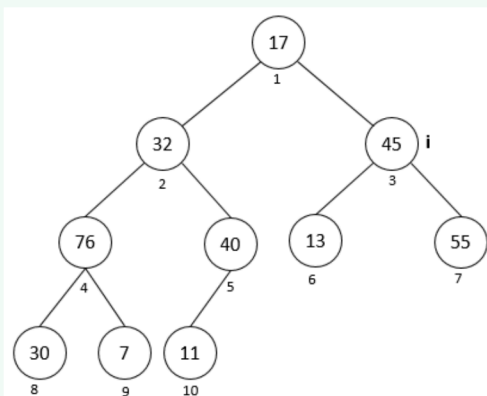
Check for index $i = 5$



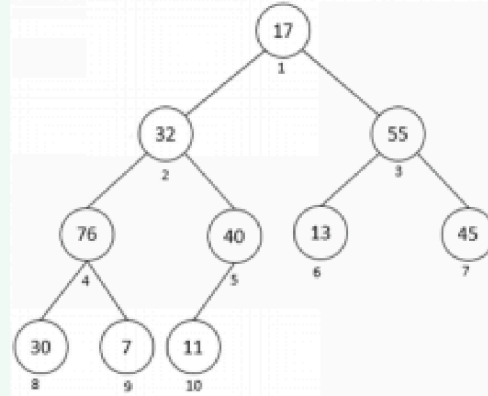
Check for index $i = 4$

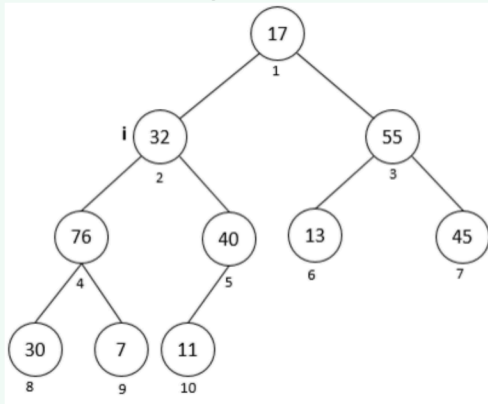
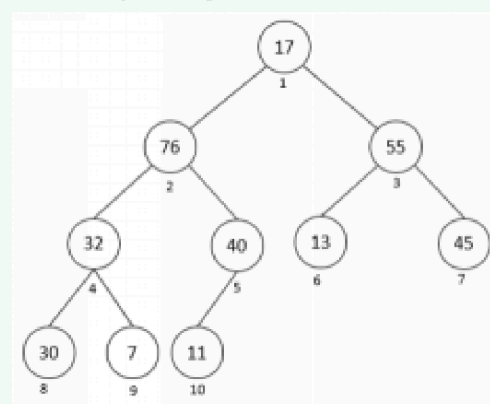
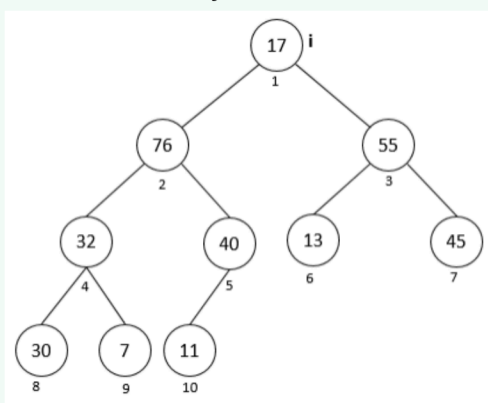
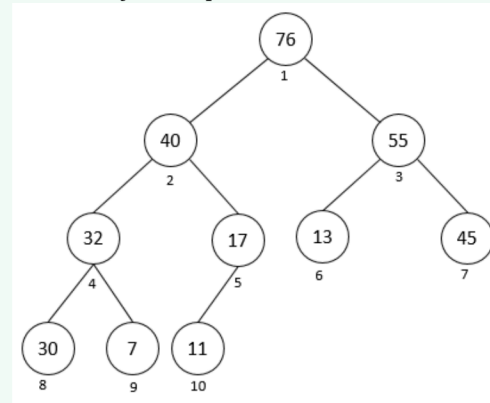


Check for index $i = 3$

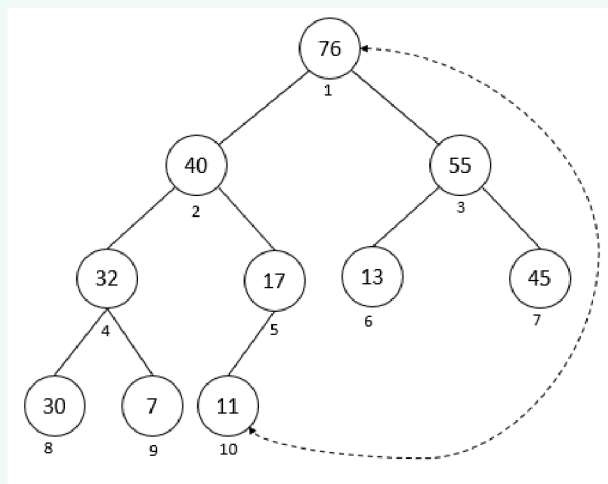


After swap at index $i = 3$

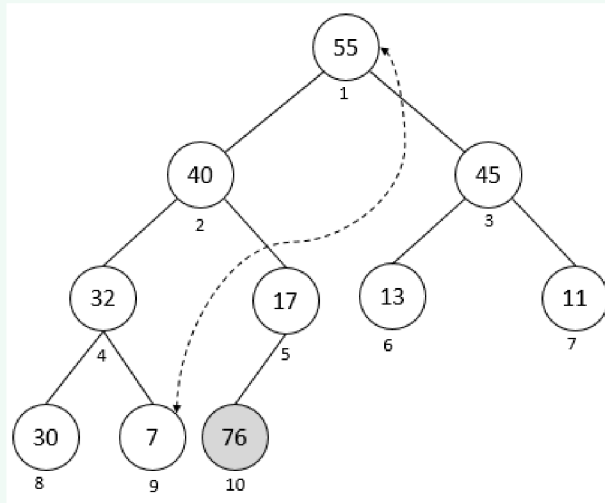


Check for index $i = 2$ After swap at index $i = 2$ Check for index $i = 1$ After swap at index $i = 1$ **Stage 2: Sorting (repeatedly swap max and heapify)**

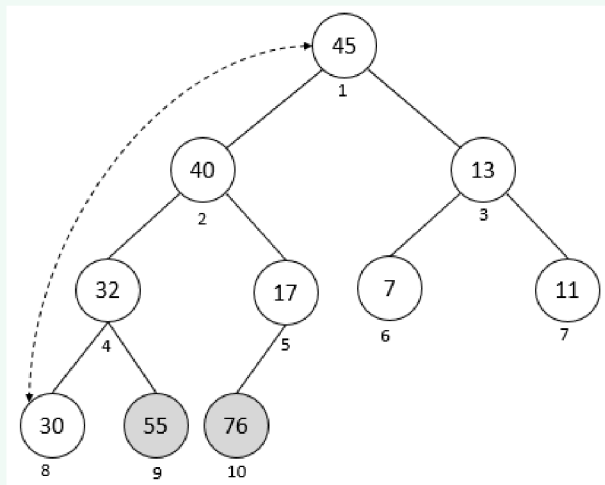
Step 1: Swap $A[1]$ with $A[10]$, and heapify the tree again.



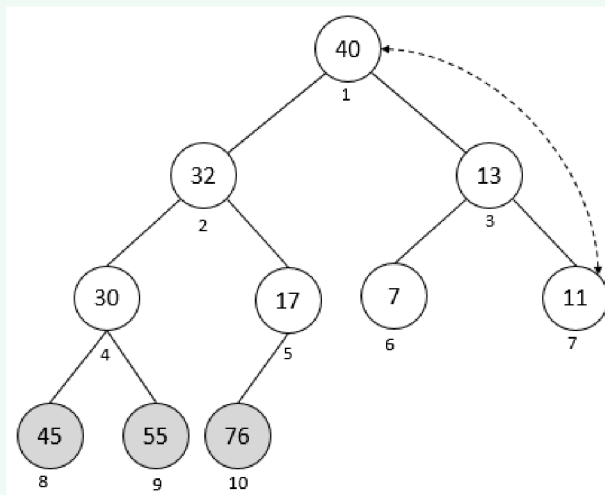
Step 2: Swap $A[1]$ with $A[9]$, and heapify the tree again.



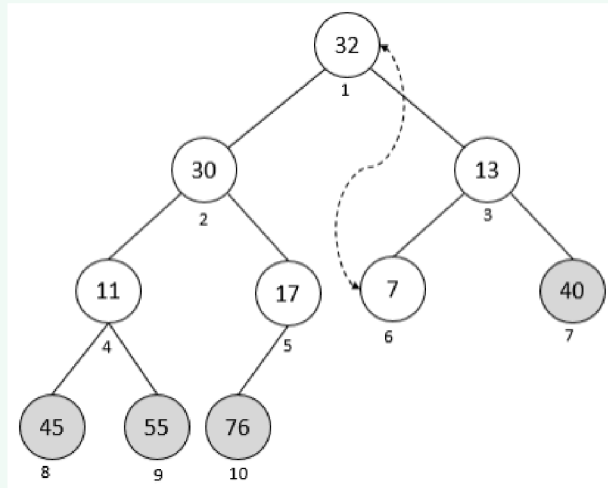
Step 3: Swap $A[1]$ with $A[8]$, and heapify the tree again.



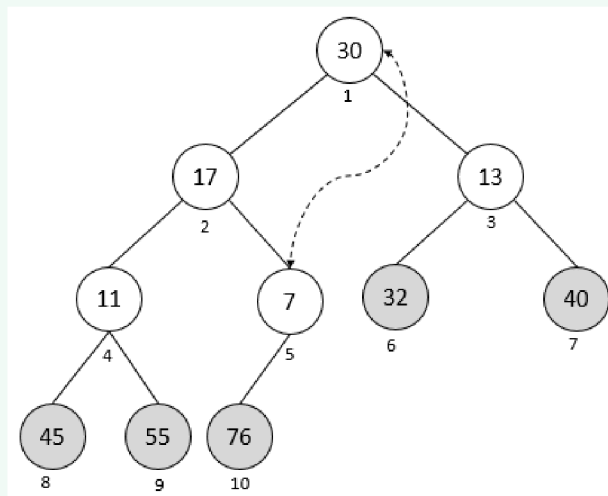
Step 4: Swap $A[1]$ with $A[7]$, and heapify the tree again.



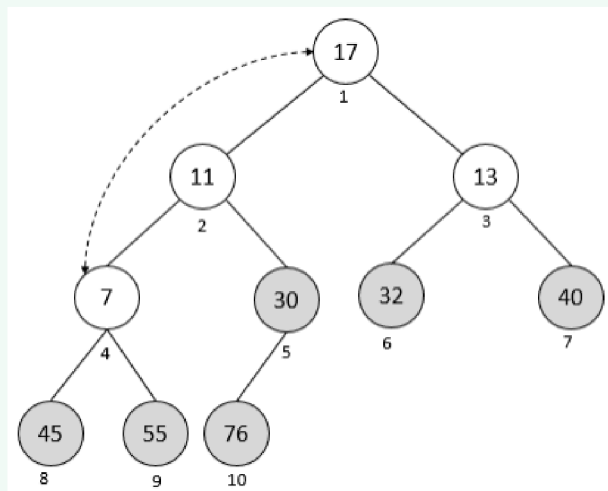
Step 5: Swap $A[1]$ with $A[6]$, and heapify the tree again.



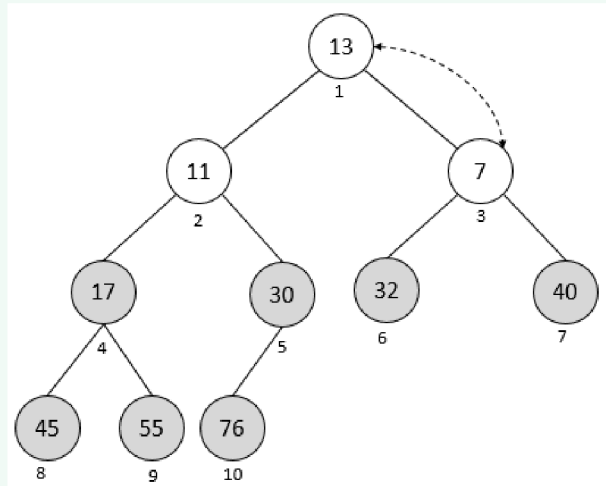
Step 6: Swap $A[1]$ with $A[5]$, and heapify the tree again.



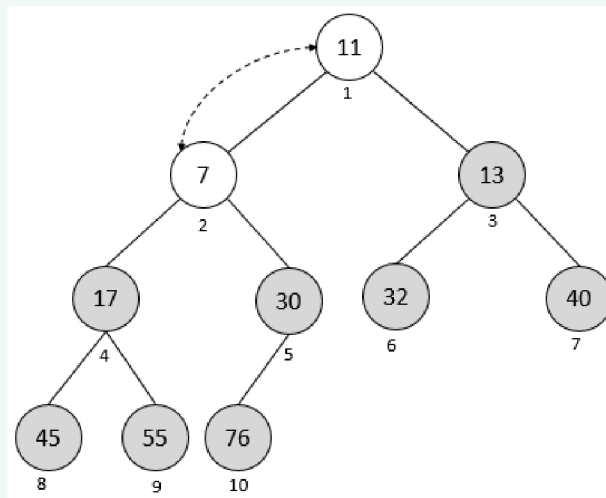
Step 7: Swap $A[1]$ with $A[4]$, and heapify the tree again.



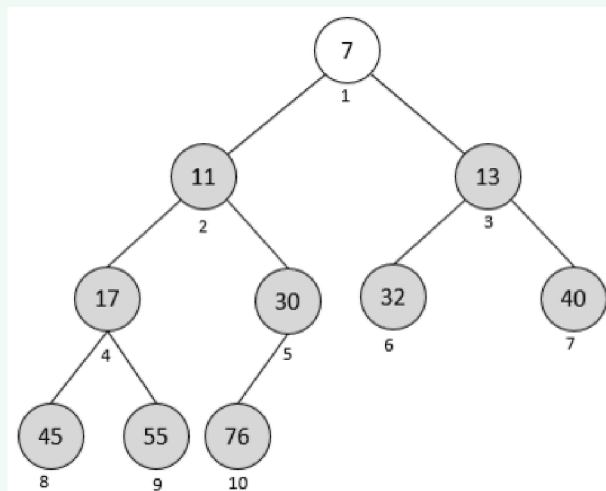
Step 8: Swap $A[1]$ with $A[3]$, and heapify the tree again.



Step 9: Swap $A[1]$ with $A[2]$, and heapify the tree again.



Step 10: Read elements in BFS order. They are sorted.



3.7 Order Statistics

Definition 3.7.1: Order Statistics

Given an array $A = [a_1, a_2, a_3, \dots, a_n]$, the i -th **order statistic** of A is the i -th smallest element in A when the array is sorted in non-decreasing order.

- If $i = 1$: 1st order statistic = smallest element (minimum).
- If $i = n$: n -th order statistic = largest element (maximum).
- If $i = n/2$: median (middle element).

Example 3.7.1 (Order Statistics Example)

Let $A = [7, 2, 9, 4, 1]$. Sorted array: $[1, 2, 4, 7, 9]$.

1st order statistic = 1,

3rd order statistic = 4,

5th order statistic = 9.

3.7.1 Naive Approach

The simplest method is to sort the array and then pick the element at index i (1-based).

Naive Order Statistic

```

1 NAIVE-ORDER-STATISTIC(A, n, i):
2     sort A in non-decreasing order
3     return A[i]           # assuming 1-indexed

```

Time Complexity

- Sorting: $O(n \log n)$ (using comparison-based sort).
- Returning the element: $O(1)$.

$$T_{\text{naive}}(n) = O(n \log n)$$

Note:-

The naive approach sorts the entire array even though we only need a single element. For very large arrays this is expensive. For example, to find the minimum element we can do a simple linear scan in $O(n)$ time; sorting is unnecessary. This motivates faster selection algorithms.

3.7.2 QuickSelect (Deterministic)

QuickSelect is based on the partition idea of QuickSort, but it recurses only on the side that contains the desired order statistic.

3.7.3 Algorithm

Algorithm 3.7.1

QUICK_SELECT(*A*, *low*, *high*, *i*):

Input: Array *A*, indices *low* and *high*, integer *i* (1-based rank)

Output: The *i*-th smallest element in *A*[*low*..*high*]

1. If *low* = *high*: return *A*[*low*]
2. If *low* < *high*:
 - (a) *p* = QUICK_PARTITION(*A*, *low*, *high*) (random pivot)
 - (b) *k* = *p* - *low* + 1
 - (c) If *k* = *i*: return *A*[*p*]
 - (d) If *i* < *k*: return QUICK_SELECT(*A*, *low*, *p* - 1, *i*)
 - (e) If *i* > *k*: return QUICK_SELECT(*A*, *p* + 1, *high*, *i* - *k*) (if *k* = 7 and *i* = 9, then you need 9 - 7 = 2nd smallest element in right subarray)

The partition step is the same as in QuickSort: it rearranges the array so that elements smaller than the pivot come before it, and larger after it.

3.7.4 Complexity Analysis of QuickSelect

Best Case

The pivot divides the array into two equal halves. The algorithm recurses on only one side.

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \implies T_{\text{best}}(n) = O(n)$$

Worst Case

The pivot is always the smallest or largest element, producing a partition of sizes 0 and $n - 1$.

$$T(n) = T(n - 1) + O(n) \implies T_{\text{worst}}(n) = O(n^2)$$

Average Case

On average, the pivot is close to the middle. Even if the partition is not perfectly balanced, say one side is $8n/10$ and the other $n/10$, we recurse on the larger side:



$$T(n) = T\left(\frac{8n}{10}\right) + O(n) = O(n)$$

Thus the average case is linear: $T_{\text{avg}}(n) = O(n)$.

3.7.5 Selection in Expected Linear Time

Randomized QuickSelect (RandSelect)

To avoid the worst $O(n^2)$ behaviour, we randomize the choice of pivot.

Algorithm**Algorithm 3.7.2****RAND_SELECT(A, low, high, i):****Input:** Array A , indices low and $high$, integer i (1-based rank)**Output:** The i -th smallest element in $A[low..high]$

1. If $low = high$: return $A[low]$
2. If $low < high$:
 - (a) $p = \text{RAND_PARTITION}(A, low, high)$ (random pivot)
 - (b) $k = p - low + 1$
 - (c) If $k = i$: return $A[p]$
 - (d) If $i < k$: return $\text{RAND_SELECT}(A, low, p - 1, i)$
 - (e) If $i > k$: return $\text{RAND_SELECT}(A, p + 1, high, i - k)$ (if $k = 7$ and $i = 9$, then you need $9 - 7 = 2$ nd smallest element in right subarray)

Expected Time Complexity

- **Best case:** The best case happens when, at **every** recursive call, the chosen pivot splits the array as evenly as possible — i.e., the pivot lands at (or near) the middle.

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \implies T_{\text{best}}(n) = O(n)$$

- **Worst case:** Still theoretically $O(n^2)$ if we are extremely unlucky (all random pivots are extreme). However, the probability of this happening is exponentially small. As, random pivot $\rightarrow$ prevents consistently bad splits. But, if the pivot always isolates one element, the recursive subproblem has size $n - 1$:

$$T(n) = T(n - 1) + O(n)$$

By solving this we get:

$$T_{\text{worst}}(n) = O(n^2)$$

- **Expected (average) case:** $O(n)$. In Randomized QuickSelect, every element has equal probability of being chosen as pivot. The recurrence for the expected number of comparisons is:

$$E[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} E[T(\max(k, n - k - 1))] + O(n)$$

Solving this gives $E[T(n)] = O(n)$.

$$E[T_{\text{rand}}(n)] = O(n)$$

Note:-

In practice, randomized QuickSelect is extremely fast and efficient. The probability of hitting the worst case is negligible, especially for large n .

3.8 Selection in Worst-Case Linear Time (Median of Medians)

Definition 3.8.1: Median of Medians

This deterministic algorithm finds the i -th smallest element of an input array of n elements in $O(n)$ time in the worst case, by using a carefully chosen pivot (the median of medians) that guarantees a good partition. Randomized QuickSelect has $O(n^2)$ worst case. The **Median of Medians** algorithm (Blum, Floyd, Pratt, Rivest, Tarjan) finds the i -th order statistic in **worst-case linear time** $O(n)$. It chooses a pivot that guarantees a good split (at least 30% of elements on each side).

3.8.1 Algorithm Steps

1. Divide the n elements of the input array into $\lceil n/5 \rceil$ groups of at most 5 elements each (the last group may have fewer than 5).
2. Find the median of each group by sorting the group (e.g., using insertion sort) and taking the middle element. For groups with an even number of elements, take the larger of the two medians.
3. Recursively apply the selection algorithm to find the median x of the $\lceil n/5 \rceil$ medians (the “median of medians”).
4. Partition the input array around x using a modified version of PARTITION, letting k be the number of elements on the low side (elements $\leq x$).
5. Recursively select the i -th smallest element:
 - If $i \leq k$, recurse on the low side.
 - Else, recurse on the high side to find the $(i - k)$ -th smallest element.

Main Idea

The goal is to choose a pivot that is neither too small nor too large. Median of Medians guarantees a *good split*, so a large fraction of elements are discarded in every recursive step.

Step 1: Divide into Groups of 5

Suppose the input array contains n elements.

Divide the elements into groups of 5:

$$\frac{n}{5} \text{ groups}$$

Example:

$$[2, 9, 1, 7, 5], [8, 3, 6, 4, 10], [15, 11, 12, 14, 13]$$

Step 2: Find Median of Each Group

Sort each group and take the middle element.

$$[1, 2, 5, 7, 9] \rightarrow 5$$

$$[3, 4, 6, 8, 10] \rightarrow 6$$

$$[11, 12, 13, 14, 15] \rightarrow 13$$

So the medians are:

$$[5, 6, 13]$$

Step 3: Find the Median of Medians

Find the median of the medians: [5, 6, 13]

Its median is: $x = 6$

This value x is used as the pivot.

Why is this Pivot Good?

Since x is the median of the medians, at least half of the medians are $\leq x$

Number of medians: $\frac{n}{5}$

Half of them:

$$\frac{1}{2} \cdot \frac{n}{5} = \frac{n}{10}$$

Thus at least $\frac{n}{10}$ groups have median $\leq x$.

Important Observation

Each group contains 5 elements.

After sorting a group: $[a, b, m, c, d]$

If the median $m \leq x$, then: $a, b, m \leq x$

So each such group contributes at least 3 elements that are $\leq x$.

How Many Elements are Guaranteed?

Number of such groups:

$$\frac{n}{10}$$

Each contributes at least 3 elements:

$$3 \times \frac{n}{10} = \frac{3n}{10}$$

Therefore:

$$\frac{3n}{10} \text{ elements are guaranteed to be } \leq x$$

Similarly,

$$\frac{7n}{10} \text{ elements are guaranteed to be } \geq x$$

Size of the Largest Recursive Subproblem

Total elements: n

At least $\frac{3n}{10}$ elements are discarded. i.e., at least these elements are definitely not needed in the next recursive call.

So the largest remaining side contains at most:

$$n - \frac{3n}{10} = \frac{7n}{10}$$

Hence the recursive subproblem size is at most:

$$\frac{7n}{10}$$

Recurrence Relation

- Finding medians of groups takes linear time:

$$\frac{n}{5} * O(1) = O(n)$$

$$O(n)$$

- Finding the median of medians:

$$T\left(\frac{n}{5}\right)$$

- Recursive selection after partition:

$$T\left(\frac{7n}{10}\right)$$

Therefore,

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

Why Does This Become Linear?

Notice:

$$\frac{n}{5} + \frac{7n}{10} = \frac{2n}{10} + \frac{7n}{10} = \frac{9n}{10}$$

So the total recursive work is on only:

$$\frac{9n}{10}$$

elements, not the full n .

Thus the recursion shrinks quickly.

Solving the Recurrence

Start with the recurrence:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

Solving this recurrence relation gives: Hence,

$$T(n) = O(n)$$

Final Intuition

Median of Medians guarantees that at least 30% of elements are discarded in every step.

Therefore the recursion never becomes highly unbalanced, giving worst-case linear time.

3.8.2 Comparison

Algorithm	Best Case	Average Case	Worst Case
Naive (sorting)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Deterministic QuickSelect	$O(n)$	$O(n)$	$O(n^2)$
Randomized QuickSelect	$O(n)$	$O(n)$	$O(n^2)$ (prob. negligible)
Median of Medians	$O(n)$	$O(n)$	$O(n)$

Note:-

In practice, randomized QuickSelect is usually preferred because of its simplicity and excellent average performance. The Median of Medians is mostly of theoretical interest, though it can be used when worst-case guarantees are required.

3.9 Assignment

Question 3.4

1. Write the algorithm for Binary Search. Give its time and space complexity with brief reasoning.
2. Write the algorithm for Min Max Finding. Give its time and space complexity with brief reasoning.
3. Write the algorithm for Merge Sort. Give its time and space complexity with brief reasoning.
4. Write the algorithm for Quick Sort. Give its time and space complexity with brief reasoning.
5. Write the algorithm for Heap Sort. Give its time and space complexity with brief reasoning.
6. Write the algorithm for Randomized Quick Sort. Give its expected time and space complexity with brief reasoning.
7. Define Order Statistics. Explain Selection in Expected Linear Time (Randomized QuickSelect) with reasoning.
8. Explain Selection in Worst-case Linear Time (Median of Medians) with reasoning.